

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

Домашняя работа №5

Выполнила:

Красноперова Е. Д.

Группа: К3443

Проверил:

Добряков Д. И.

Санкт-Петербург,

2026 г.

Задание

Подключить и настроить RabbitMQ/Kafka и реализовать межсервисное взаимодействие посредством очередей сообщений.

Исходная архитектура

За основу взята ЛР2: API Gateway (3000), UserService (3001), RecipeService (3002) и SocialService (3003). Каждый бизнес-сервис использует собственную PostgreSQL database. Существующее синхронное HTTP REST-взаимодействие сохранено для операций, где ответ требуется непосредственно клиенту.

Подключение RabbitMQ

Для очередей сообщений выбран RabbitMQ. Используется topic exchange `recipes.events`. Сообщения публикуются как persistent, очереди являются durable. Подключение задаётся переменной RABBITMQ_URL, по умолчанию `amqp://guest:guest@localhost:5672`. Для локального запуска добавлен docker-compose.yml с RabbitMQ Management.

Реализованные события

- `recipe.deleted` - RecipeService публикует событие после успешного удаления рецепта. SocialService получает его и удаляет связанные likes, comments и saved_recipes.
- `recipe.social.updated` - SocialService публикует событие после изменения лайка, сохранения или комментария. RecipeService получает событие и сохраняет агрегированную статистику в своей БД в таблице `recipe_social_stats`.

Очереди

- `recipe-service.social-updates` - очередь `RecipeService`; подписана на `recipe.social.updated` и `recipe.deleted`.
- `social-service.recipe-events` - очередь `SocialService`; подписана на `recipe.deleted`.

Обработка сообщений

Сообщения разбираются как JSON. После успешной обработки `consumer` подтверждает сообщение через ACK. При ошибке обработки используется NACK без повторной постановки сообщения в очередь. Очереди и `exchange` создаются автоматически при старте сервисов.

Проверка работы

После запуска `RabbitMQ` и двух сервисов в логах появляются сообщения о запуске `consumer`. Операция `like/save/comment` приводит к публикации `recipe.social.updated`, а состояние агрегированной статистики можно проверить через защищённый внутренний `endpoint` `RecipeService` `/internal/recipes/{recipe_id}/social-stats`.

При удалении рецепта `RecipeService` публикует `recipe.deleted`. После обработки события `SocialService` удаляет связанные социальные записи. Результат можно проверить через `/internal/recipes/{recipe_id}/event-state` с сервисным токеном.

Вывод

В результате к микросервисной архитектуре ЛР2 добавлено асинхронное взаимодействие через `RabbitMQ`. Реализованы `publisher` и `consumer` в `RecipeService` и `SocialService`, `durable topic exchange`, очереди и обработка событий. Синхронные HTTP-вызовы ЛР2 сохранены там, где они необходимы для непосредственного ответа на пользовательский запрос.